教学事务管理系统教师端数据库报告

23120740 焦艺峰

一、项目背景

1.1 教师端业务背景

教学事务管理系统面向管理员、教师和学生三类角色，其中教师端承担授课管理、学生名单查看、成绩录入、调课申请、考试修改申请、退课审核、补考确认和通知查看等工作。教师端虽然只是系统的一部分，但它直接连接教学班、学生、成绩、考试和申请审核流程，是教学事务从“排课”走向“教学执行”的关键环节。

传统教师教学事务往往分散在纸质名单、Excel 成绩表、人工通知和线下审批中，容易出现成绩版本不一致、审批过程不可追踪、调课和考试时间冲突难以及时发现等问题。因此教师端重点讨论如何围绕教师工作流建立数据库模型，让课表、成绩、申请、通知等数据形成可查询、可校验、可追踪的系统化流程。

1.2 教师端建设目标

教师只能查看和操作本人教学班，保证教师端数据边界清晰。

教师可以按学期查看授课课表、教学班详情和学生名单。

成绩录入支持暂存、提交、批量导入导出，并由数据库触发器统一计算总评和绩点。

调课、考试修改、成绩修改等高风险操作必须通过申请表保存过程，由管理员审核后生效。

退课审核、补考确认和通知中心让教师能够参与跨角色事务闭环。

二、总体功能

教师端属于教学事务管理系统总体功能的一部分。系统总体上仍包含基础数据管理、学生选课与学业服务、教师教学事务、审批与考试管理、系统支撑五类功能。保留总体功能图可以说明教师端与其它模块的关系，如图 1 所示。

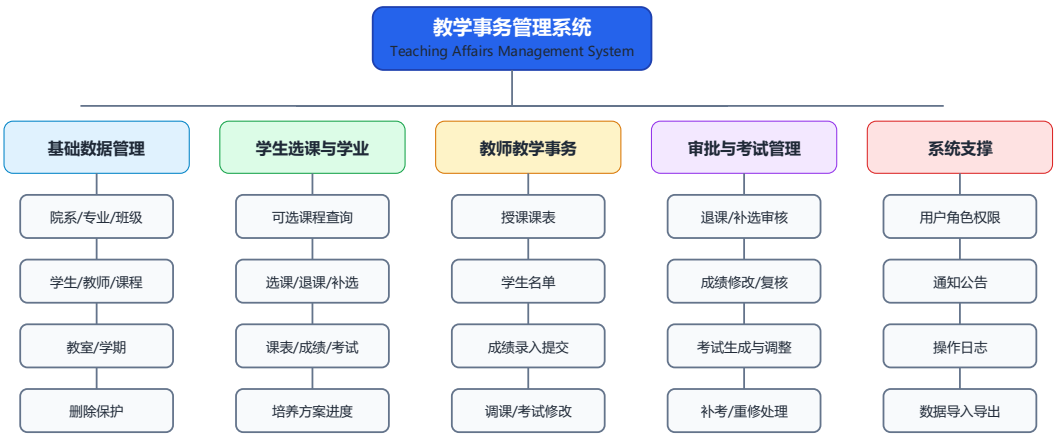


图 1 教学事务管理系统总体功能设计图

在总体功能中，教师端主要位于“教师教学事务”模块，同时与“审批与考试管理”“系统支撑”紧密关联。教师端并不直接维护学期、课程、学生等全局数据，而是读取这些基础数据，并围绕本人教学班完成教学执行。

三、教师端功能设计

3.1 授课课表与教学班详情

3.1.1 前端设计

授课管理页面使用周课表网格展示教师课程。横向为星期，纵向为节次，课程卡片展示课程名称、节次、周次、单双周、教室和选课人数。

页面支持按学期切换课表，默认优先显示当前学期。课表状态只描述学期状态：当前学期显示“当前”，历史学期显示“已结束”，未来学期显示“未开始”。

点击课程卡片打开教学班详情抽屉，展示课程编号、课程类型、学分、考核方式、平时考试比例、教室、容量和学生名单。

3.1.2 后端设计

教师教学班列表接口默认只返回当前学期，也支持 includeAllSemesters 参数查询全部学期，供课表页切换学期。

后端根据当前登录用户关联的教师工号查询 teaching_class.teacher_id，保证教师只能看到本人教学班。

教学班详情和学生名单接口都会校验教学班归属，防止教师通过接口参数访问其它教师数据。

3.1.3 数据库落点

课表数据主要来自 teaching_class、teaching_class_session、course、semester、classroom。

学生名单通过 course_selection 和 student 联查，只展示 selection_status 为已选的学生。

教师端权限边界来自 teaching_class.teacher_id，不额外建立教师权限映射表。

3.1.4 设计原因

教师最常用的操作是查看本人在某学期的课程安排，因此课表视图比普通列表更符合使用习惯。教学班作为核心实体，把课程、教师、学期、教室、学生名单、成绩和考试串在一起，教师端的大部分功能都从教学班出发。

3.2 成绩录入与成绩修改申请

3.2.1 前端设计

成绩页面采用可编辑表格，教师录入平时成绩和考试成绩。

页面支持单条暂存、单条提交、批量暂存、批量提交、CSV/XLSX 导出和离线导入。

页面展示统计卡片和绩点分布，方便教师在提交前检查平均分、最高分、最低分、及格率和成绩分布。

已提交成绩只读，如果需要修改，教师只能提交成绩修改申请。

3.2.2 后端设计

保存成绩时校验教学班是否属于当前教师。

暂存成绩不受成绩提交时间限制，正式提交成绩必须处于 semester.score_submit_start 和 semester.score_submit_end 范围内。

成绩修改申请保存原平时成绩、原考试成绩、原总评成绩、原绩点和申请原因，避免审核时原始数据丢失。

3.2.3 数据库落点

成绩写入 score。usual_score 和 exam_score 由教师录入，final_score 和 gpa_point 由数据库触发器计算。

成绩修改申请写入 score_change_request。管理员审核通过后，将对应 score 退回未提交状态，教师再重新修改。

3.2.4 设计原因

成绩属于高风险数据。系统允许教师先暂存，降低一次性录入压力；正式提交受学期时间窗口控制，保证教学管理节奏；提交后不能直接修改，而是通过申请表保留修改原因和审核轨迹。

3.3 调课申请

3.3.1 前端设计

教师选择教学班后，页面自动带出原星期、原节次、原周次和原教室。

申请表限制周次不可修改，节次选择根据原节次跨度进行限制。目标教室根据申请后的星期和节次动态加载。

申请记录列表展示申请状态、申请内容、审核意见和撤销入口。待审核申请可以撤销。

3.3.2 后端设计

提交申请时校验教学班归属、周次不变、节次跨度一致。

后端检查目标教室在同学期、同星期、同节次、同单双周范围内是否被占用。

后端还检查教师本人在目标时间是否已有其它课程，避免同一教师课程冲突。

3.3.3 数据库落点

调课申请写入 schedule_adjust_request，保存原星期、原节次、原周次、原教室，以及申请后的星期、节次、周次和教室。

审核通过后才更新 teaching_class_session 和 teaching_class 的教室信息。

3.3.4 设计原因

调课会影响学生课表、教师时间和教室占用，不能由教师直接修改主表。申请表将“教师提出变更”和“管理员确认生效”分离，既保留过程，也降低排课冲突风险。

3.4 考试安排与考试修改申请

3.4.1 前端设计

考试页面展示当前学期本人课程的考试日期、开始时间、结束时间、教室和考试状态。

教师可以提交考试时间或教室修改申请。选择申请后日期和时间后，教室下拉动态显示可用教室。

考试修改申请列表展示原安排、申请后安排、申请原因、审核状态和审核备注。

3.4.2 后端设计

后端校验考试所属教学班是否由当前教师授课。

同一考试存在待审核修改申请时不能重复提交。

提交申请和审核通过时均检查目标教室考试时间冲突和教师本人考试时间冲突。

3.4.3 数据库落点

考试安排保存在 exam_arrangement。

考试修改申请保存在 exam_change_request，包含原考试日期、原时间、原教室、申请后日期、申请后时间和申请后教室。

3.4.4 设计原因

考试安排直接影响学生和教室资源，因此教师只能提出修改申请。系统用 exam_arrangement 表表示最终考试安排，用 exam_change_request 表保存尚未生效的变更请求。

3.5 退课审核、补考确认与通知

3.5.1 前端设计

退课审核页面展示学生、教学班、课程、退课原因、教师审核状态和管理员审核状态。

补考确认页面展示学生补考申请、课程、成绩、申请原因和安排信息。

通知中心展示公告、审批结果和系统消息，支持查看详情和标记已读。

3.5.2 后端设计

教师只能审核本人教学班相关退课申请。

退课最终通过后，系统更新 course_selection 状态并清理对应 score，避免退课学生继续保留课程成绩。

通知接口按当前登录用户查询 notification，避免教师查看其它用户消息。

3.5.3 数据库落点

退课申请来自 course_drop_request，补考申请来自 makeup_exam_request，通知来自 notification。

公告正文来自 announcement，通知表通过 announcement_id 或 ref_type、ref_id 关联具体业务。

3.5.4 设计原因

退课和补考都需要教师参与，但最终往往还涉及管理员安排和状态确认。独立申请表可以保存教师意见和管理员意见，通知表则把处理结果主动推送给教师。

四、数据库设计

4.1 概念设计

以下的概念设计描述保留了系统通用实体，包括院系、专业、行政班、学生、教师、用户、角色、学期、课程、教室、教学班、上课时间、选课、成绩、考试安排、申请、公告、通知和操作日志等。但该报告将重点关注这些实体中与教师工作流直接相关的部分。

教师端的主要业务的 ER 图如图 2 所示，展示了教师端处理的主要业务相关的实体与其它们之间的关系。而教师端完整概念模型以教师和教学班为中心，其完整实体关系拆成如下两张 UML 表

示法的 ER 图：图 3 展示授课、选课、成绩和考试等核心教学数据关系；图 4 展示调课、退课、成绩修改、考试修改、补考和通知等申请流转关系。

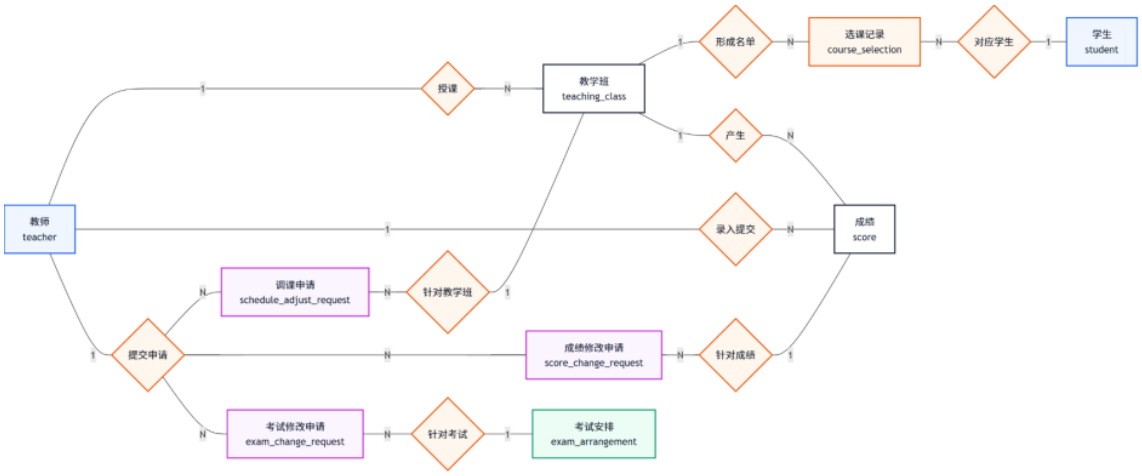


图 2 教师端主要业务 ER 图

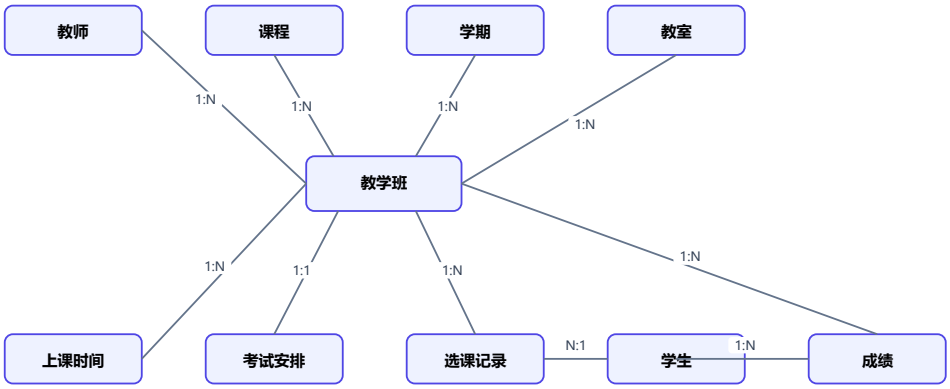


图 3 教师端核心教学数据 ER 图

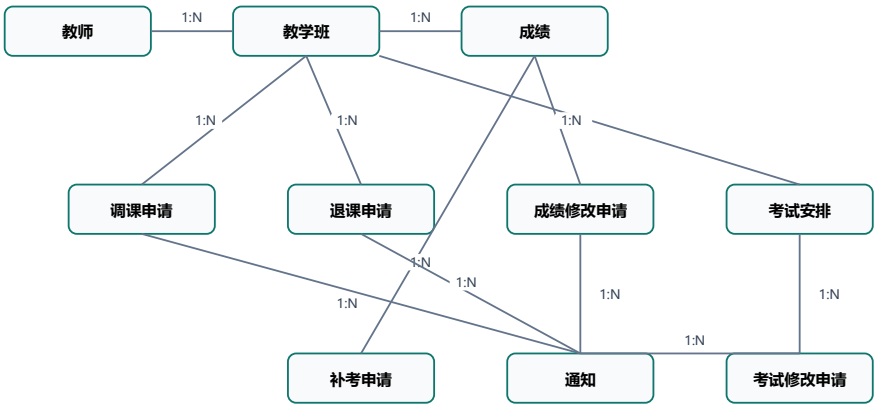


图 4 教师端申请流转 ER 图

4.2 逻辑设计

逻辑设计采用关系模型。学生、教师、课程、院系、专业、行政班、教室使用业务编号作为主键；学期、教学班、选课、成绩、考试、申请、通知和日志使用自增主键。系统设置外键维护引用完整性，并在选课、成绩、考试安排等表上设置唯一约束，避免重复记录。

逻辑设计的核心思路是把稳定的基础资料、可反复发生的业务记录、需要审核流转的申请记录分开建模。基础资料表主要保存相对稳定的对象，例如学生、教师、课程、教室；业务记录表保存教学运行过程中持续产生的数据，例如教学班、选课、成绩和考试；申请审批表保存流程状态、原始值、申请值和审核信息。这样设计可以降低表之间的耦合，也便于后续分别扩展基础维护、教学运行和审批流程。

教学班采用主表加明细表的方式设计。teaching_class 保存一个教学班的课程、教师、学期、教室、容量和周次，teaching_class_session 保存星期、节次和单双周。这样比把多个星期和节次直接塞进教学班主表更规范，也能支持一门课一周多次上课。成绩表和选课表都围绕教学班建立，保证课表、成绩、考试和申请都能回到同一个教学实体上。

表 1 主要数据表清单

类别	表名	说明
基础数据	department、major、class_info、student、teacher	院系、专业、班级、学生、教师资料
权限支撑	sys_user、sys_role、sys_user_role	登录账号和角色关系
教学资源	semester、course、classroom、teaching_class、teaching_class_session	学期、课程、教室、教学班和上课时间
学习过程	course_selection、student_course_plan、score、exam_arrangement	选课、计划、成绩和考试
申请审批	course_selection_request、course_drop_request、schedule_adjust_request、score_change_request、score_review_request、exam_change_request、makeup_exam_request、course_retake_request	各类跨角色申请
通知日志	announcement、notification、operation_log	公告、通知和操作日志

表 2 department 表结构

字段名	类型长度	约束	说明
department_id	VARCHAR(10)	PK	院系编号
department_name	VARCHAR(50)	UNIQUE	院系名称
office_address	VARCHAR(100)	NULL	办公地点
contact_phone	VARCHAR(20)	NULL	联系电话

表 3 student 表结构

字段名	类型长度	约束	说明
student_id	VARCHAR(12)	PK	学号

student_name	VARCHAR(20)	NOT NULL	姓名
gender	CHAR(1)	NOT NULL	性别
birth_date	DATE	NULL	出生日期
hometown	VARCHAR(50)	NULL	籍贯
phone	VARCHAR(20)	NULL	手机号
email	VARCHAR(50)	NULL	邮箱
enrollment_year	CHAR(4)	NOT NULL	入学年份
department_id	VARCHAR(10)	FK	院系编号
major_id	VARCHAR(10)	FK	专业编号
class_id	VARCHAR(10)	FK	班级编号
status	VARCHAR(20)	DEFAULT '正常'	学籍状态

表 4 teacher 表结构

字段名	类型长度	约束	说明
teacher_id	VARCHAR(12)	PK	工号
teacher_name	VARCHAR(20)	NOT NULL	姓名
gender	CHAR(1)	NOT NULL	性别
birth_date	DATE	NULL	出生日期
title_name	VARCHAR(20)	NOT NULL	职称
education	VARCHAR(20)	NULL	学历
phone	VARCHAR(20)	NULL	手机号
email	VARCHAR(50)	NULL	邮箱
department_id	VARCHAR(10)	FK	所属院系
status	VARCHAR(20)	DEFAULT '在职'	教师状态

表 5 semester 表结构

字段名	类型长度	约束	说明
semester_id	INT	PK, AI	学期编号
semester_name	VARCHAR(30)	UNIQUE	学期名称
school_year	VARCHAR(20)	NOT NULL	学年
term_code	VARCHAR(10)	NOT NULL	学期代码
start_date	DATE	NOT NULL	开始日期
end_date	DATE	NOT NULL	结束日期
course_select_start	DATETIME	NOT NULL	选课开始

course_select_end	DATETIME	NOT NULL	选课结束
course_selection_manual_mode	VARCHAR(10)	DEFAULT 'AUTO'	选课控制
score_submit_start	DATETIME	NULL	成绩提交开始
score_submit_end	DATETIME	NULL	成绩提交结束
makeup_request_start	DATETIME	NULL	补考申请开始
makeup_request_end	DATETIME	NULL	补考申请结束
is_current	TINYINT	DEFAULT 0	当前学期

表 6 course 表结构

字段名	类型长度	约束	说明
course_id	VARCHAR(12)	PK	课程编号
course_name	VARCHAR(50)	UNIQUE	课程名称
course_type	VARCHAR(20)	NOT NULL	课程类型
credit	DECIMAL(4,1)	NOT NULL	学分
total_hours	INT	NOT NULL	总学时
assessment_method	VARCHAR(20)	DEFAULT '考试'	考核方式
usual_ratio	TINYINT	DEFAULT 40	平时占比
exam_ratio	TINYINT	DEFAULT 60	考试占比
department_id	VARCHAR(10)	FK	开课院系
is_required	TINYINT	DEFAULT 1	是否必修
description	VARCHAR(200)	NULL	课程说明

表 7 teaching_class 表结构

字段名	类型长度	约束	说明
teaching_class_id	INT	PK, AI	教学班编号
semester_id	INT	FK	学期编号
course_id	VARCHAR(12)	FK	课程编号
teacher_id	VARCHAR(12)	FK	教师工号
classroom_id	VARCHAR(10)	FK, NULL	教室编号
course_start_date	DATE	NULL	开课日期
week_range	VARCHAR(30)	NOT NULL	周次范围
is_retake_class	TINYINT	DEFAULT 0	是否重修班
max_students	INT	DEFAULT 60	容量
selected_count	INT	DEFAULT 0	已选人数

status	VARCHAR(20)	DEFAULT '未开始'	教学班状态

表 8 session 表结构

字段名	类型长度	约束	说明
session_id	INT	PK, AI	上课时间编号
teaching_class_id	INT	FK	教学班编号
week_day	TINYINT	NOT NULL	星期几
section_desc	VARCHAR(20)	NOT NULL	节次
week_parity	VARCHAR(10)	DEFAULT '全周'	全周/单周/双周
sort_order	INT	DEFAULT 1	排序

表 9 selection 表结构

字段名	类型长度	约束	说明
selection_id	BIGINT	PK, AI	选课记录编号
student_id	VARCHAR(12)	FK	学号
teaching_class_id	INT	FK	教学班编号
selection_time	DATETIME	CURRENT_TIMESTAMP	选课时间
selection_status	VARCHAR(20)	DEFAULT '已选'	选课状态

表 10 score 表结构

字段名	类型长度	约束	说明
score_id	BIGINT	PK, AI	成绩编号
student_id	VARCHAR(12)	FK	学号
teaching_class_id	INT	FK	教学班编号
usual_score	DECIMAL(5,2)	NULL	平时成绩
exam_score	DECIMAL(5,2)	NULL	考试成绩
final_score	DECIMAL(5,2)	NULL	总评成绩
gpa_point	DECIMAL(3,1)	NULL	绩点
is_submitted	TINYINT	DEFAULT 0	是否提交
entered_at	DATETIME	NULL	录入时间

表 11 exam 表结构

字段名	类型长度	约束	说明
exam_id	BIGINT	PK, AI	考试安排编号

teaching_class_id	INT	FK, UNIQUE	教学班编号
exam_date	DATE	NOT NULL	考试日期
start_time	TIME	NOT NULL	开始时间
end_time	TIME	NOT NULL	结束时间
classroom_id	VARCHAR(10)	FK, NULL	考试教室
exam_status	VARCHAR(20)	DEFAULT '已排定'	考试状态
created_at	DATETIME	CURRENT_TIMESTAMP	创建时间
updated_at	DATETIME	NULL	更新时间

表 12 申请审批表结构摘要

表名	主键	主要字段	作用
course_selection_request	request_id BIGINT	student_id、teaching_class_id、request_reason、request_status、reviewed_by	补选申请
course_drop_request	drop_request_id BIGINT	student_id、teaching_class_id、teacher_status、admin_status	退课申请
schedule_adjust_request	adjust_request_id BIGINT	teaching_class_id、teacher_id、original_section_desc、requested_section_desc	调课申请
score_change_request	score_change_request_id BIGINT	score_id、teacher_id、original_usual_score、original_exam_score	教师成绩修改申请
score_review_request	request_id BIGINT	student_id、score_id、request_reason、request_status	学生成绩复核申请
exam_change_request	exam_change_request_id BIGINT	exam_id、teacher_id、requested_exam_date、requested_start_time	考试修改申请
makeup_exam_request	request_id BIGINT	student_id、score_id、teacher_status、admin_status、classroom_id	补考申请
course_retake_request	request_id BIGINT	student_id、score_id、request_status	重修申请

表 13 支撑表结构摘要

表名	字段设计	作用
announcement	announcement_id BIGINT、title VARCHAR(100)、content VARCHAR(1000)、target_scope VARCHAR(20)、published_by INT	公告发布

notification	notification_id BIGINT、user_id INT、category VARCHAR(20)、title VARCHAR(100)、is_read TINYINT、ref_type VARCHAR(30)	站内通知
operation_log	log_id BIGINT、user_id INT、operation_type VARCHAR(30)、operation_content VARCHAR(200)、operation_time DATETIME	操作日志
student_course_plan	plan_id BIGINT、student_id VARCHAR(20)、teaching_class_id INT、created_at DATETIME	学生课程计划
cultivation_plan_requirement	requirement_id BIGINT、enrollment_year INT、major_id VARCHAR(20)、course_type VARCHAR(50)、required_credit DECIMAL(5,1)	培养方案要求

4.3 物理设计

教师端物理设计重点关注教师授课数据的读取效率、成绩数据的一致性、申请数据的可追踪性和教师权限边界。数据库采用 MySQL InnoDB 引擎和 utf8mb4 字符集，利用外键、唯一约束、索引、触发器和申请表状态字段支撑教师端业务。

教师端部署时不直接连接数据库，而是通过后端服务访问数据库。数据库部署在了云服务器的自建 MySQL。云端数据库便于多端访问、集中备份和迁移，但需要配置安全组白名单、独立数据库账号和定期备份，避免成绩、选课和审批记录丢失。

表 14 建库语句示例

```
DROP DATABASE IF EXISTS school;
CREATE DATABASE school DEFAULT CHARACTER SET utf8mb4;
USE school;
SET NAMES utf8mb4;
```

4.3.1 约束与索引设计

教师端最关键的约束是教学班归属、成绩唯一性和考试唯一性。teaching_class.teacher_id 用于限制教师只能访问本人教学班；score 对 student_id 和 teaching_class_id 建立唯一约束，保证一个学生在一个教学班只有一条成绩；exam_arrangement 对 teaching_class_id 建立唯一约束，保证一个教学班最多一条考试安排。

教师端常用查询需要按 teacher_id、teaching_class_id、request_status 等字段过滤，因此教学班、成绩和申请表需要围绕这些字段设计索引。这样可以提升课表加载、成绩列表、调课申请列表、考试修改申请列表的查询效率。

表 15 建表语句示例：教学班表

```
CREATE TABLE teaching_class (
  teaching_class_id INT NOT NULL AUTO_INCREMENT COMMENT "教学班编号",
  semester_id INT NOT NULL COMMENT "学期编号",
  course_id VARCHAR(12) NOT NULL COMMENT "课程编号",
  teacher_id VARCHAR(12) NOT NULL COMMENT "任课教师工号",
```

```
  classroom_id VARCHAR(10) NULL COMMENT "教室编号",
  week_range VARCHAR(30) NOT NULL COMMENT "周次范围",
  max_students INT NOT NULL DEFAULT 60 COMMENT "容量上限",
  selected_count INT NOT NULL DEFAULT 0 COMMENT "已选人数",
  status VARCHAR(20) NOT NULL DEFAULT "未开始" COMMENT "教学班状态",
  PRIMARY KEY (teaching_class_id),
  CONSTRAINT fk_teaching_class_semester FOREIGN KEY (semester_id) REFERENCES
semester(semester_id),
  CONSTRAINT fk_teaching_class_course FOREIGN KEY (course_id) REFERENCES
course(course_id),
  CONSTRAINT fk_teaching_class_teacher FOREIGN KEY (teacher_id) REFERENCES
teacher(teacher_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT="教学班表";
```

表 16 唯一约束语句示例

```
CREATE TABLE course_selection (
  selection_id BIGINT NOT NULL AUTO_INCREMENT COMMENT "选课记录编号",
  student_id VARCHAR(12) NOT NULL COMMENT "学号",
  teaching_class_id INT NOT NULL COMMENT "教学班编号",
  selection_status VARCHAR(20) NOT NULL DEFAULT "已选" COMMENT "选课状态",
  PRIMARY KEY (selection_id),
  UNIQUE KEY uk_student_teaching_class (student_id, teaching_class_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT="选课表";
```

4.3.2 触发器设计

教师端直接使用的触发器集中在成绩表 score 上。当前系统共有两个成绩触发器：trg_score_before_insert 和 trg_score_before_update。二者分别在新增成绩和更新成绩前执行，职责一致：校验平时成绩、考试成绩是否在 0 到 100 分之间；当两项成绩都存在时，读取课程成绩比例并计算总评成绩和绩点。

触发器适合处理与单表写入强相关、必须保持一致的派生字段。本系统中的 final_score 和 gpa_point 不由教师手工填写，而是在 usual_score 和 exam_score 写入时自动推导。这样可以避免教师导入成绩时误填总评成绩，也能保证学生端、教师端和管理员端看到的总评规则一致。

表 17 成绩表触发器完整清单

触发器名称	触发时机	主要作用
trg_score_before_insert	BEFORE INSERT ON score	新增成绩前校验成绩范围，计算总评成绩和绩点，补充录入时间
trg_score_before_update	BEFORE UPDATE ON score	更新成绩前校验成绩范围，重新计算总评成绩和绩点

表 18 触发器语句：trg_score_before_insert

```
DELIMITER $$
CREATE TRIGGER trg_score_before_insert
BEFORE INSERT ON score
FOR EACH ROW
BEGIN
  DECLARE v_usual_ratio TINYINT DEFAULT 40;
  DECLARE v_exam_ratio  TINYINT DEFAULT 60;

  IF NEW.usual_score IS NOT NULL AND (NEW.usual_score < 0 OR NEW.usual_score > 100) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '平时成绩必须在 0 到 100 之间';
  END IF;

  IF NEW.exam_score IS NOT NULL AND (NEW.exam_score < 0 OR NEW.exam_score > 100) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '考试成绩必须在 0 到 100 之间';
  END IF;

  IF NEW.usual_score IS NOT NULL AND NEW.exam_score IS NOT NULL THEN
    SELECT c.usual_ratio, c.exam_ratio
    INTO v_usual_ratio, v_exam_ratio
    FROM teaching_class tc
    JOIN course c ON tc.course_id = c.course_id
    WHERE tc.teaching_class_id = NEW.teaching_class_id;

    SET NEW.final_score = ROUND(NEW.usual_score * v_usual_ratio / 100 + NEW.exam_score *
v_exam_ratio / 100, 2);
    SET NEW.gpa_point = CASE
      WHEN NEW.final_score >= 90 THEN 4.0
      WHEN NEW.final_score >= 85 THEN 3.7
      WHEN NEW.final_score >= 82 THEN 3.3
      WHEN NEW.final_score >= 78 THEN 3.0
      WHEN NEW.final_score >= 75 THEN 2.7
      WHEN NEW.final_score >= 72 THEN 2.3
      WHEN NEW.final_score >= 68 THEN 2.0
      WHEN NEW.final_score >= 66 THEN 1.7
      WHEN NEW.final_score >= 64 THEN 1.5
      WHEN NEW.final_score >= 60 THEN 1.0
      ELSE 0.0
    END;
  END IF;
```

```
    IF NEW.entered_at IS NULL THEN
        SET NEW.entered_at = NOW();
    END IF;
END$$
```

表 19 触发器语句：trg_score_before_update

```
CREATE TRIGGER trg_score_before_update
BEFORE UPDATE ON score
FOR EACH ROW
BEGIN
    DECLARE v_usual_ratio TINYINT DEFAULT 40;
    DECLARE v_exam_ratio  TINYINT DEFAULT 60;

    IF NEW.usual_score IS NOT NULL AND (NEW.usual_score < 0 OR NEW.usual_score > 100) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '平时成绩必须在 0 到 100 之间';
    END IF;

    IF NEW.exam_score IS NOT NULL AND (NEW.exam_score < 0 OR NEW.exam_score > 100) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '考试成绩必须在 0 到 100 之间';
    END IF;

    IF NEW.usual_score IS NOT NULL AND NEW.exam_score IS NOT NULL THEN
        SELECT c.usual_ratio, c.exam_ratio
        INTO v_usual_ratio, v_exam_ratio
        FROM teaching_class tc
        JOIN course c ON tc.course_id = c.course_id
        WHERE tc.teaching_class_id = NEW.teaching_class_id;

        SET NEW.final_score = ROUND(NEW.usual_score * v_usual_ratio / 100 + NEW.exam_score *
v_exam_ratio / 100, 2);
        SET NEW.gpa_point = CASE
            WHEN NEW.final_score >= 90 THEN 4.0
            WHEN NEW.final_score >= 85 THEN 3.7
            WHEN NEW.final_score >= 82 THEN 3.3
            WHEN NEW.final_score >= 78 THEN 3.0
            WHEN NEW.final_score >= 75 THEN 2.7
            WHEN NEW.final_score >= 72 THEN 2.3
            WHEN NEW.final_score >= 68 THEN 2.0
```

```
        WHEN NEW.final_score >= 66 THEN 1.7
        WHEN NEW.final_score >= 64 THEN 1.5
        WHEN NEW.final_score >= 60 THEN 1.0
        ELSE 0.0
      END;
  END IF;
END$$
```

4.3.3 教师端查询设计

教师课表查询需要联查教学班、上课时间、课程、学期、教师和教室。后端查询时以 teacher_id 为过滤条件，再按学期分组展示。学生名单查询在教学班归属校验后，联查 course_selection 和 student，只返回已选学生。

表 20 教师课表查询 SQL 示例

```
SELECT tc.teaching_class_id, c.course_name, sem.semester_name,
    sess.week_day, sess.section_desc, sess.week_parity,
    tc.week_range, cl.building_name, cl.room_name
FROM teaching_class tc
JOIN teaching_class_session sess ON sess.teaching_class_id = tc.teaching_class_id
JOIN course c ON tc.course_id = c.course_id
JOIN semester sem ON tc.semester_id = sem.semester_id
LEFT JOIN classroom cl ON tc.classroom_id = cl.classroom_id
WHERE tc.teacher_id = ?;
```

4.3.4 教师端申请表设计

教师端调课、成绩修改和考试修改都采用“申请表先记录，审核后更新主表”的设计。schedule_adjust_request 保存调课前后时间与教室；score_change_request 保存原成绩快照和申请原因；exam_change_request 保存原考试安排和申请后安排。

这种物理设计可以避免教师端未审核数据污染主业务表，也让管理员审核时能够看到完整上下文。申请表中的 request_status、review_comment、reviewed_by、reviewed_at 等字段统一表达审核状态。

五、效果演示

教师端实际运行后，教师可以围绕本人教学班完成授课查看、成绩处理和申请发起。效果演示只列出教师端相关页面和业务结果，如表所示。

表 21 教师端功能效果演示摘要

页面或功能	教师操作	演示效果
授课课表	选择学期、查看课程卡片、打开教学班详情	按周课表展示本人课程，详情抽屉展示课程信息和学生名单

成绩录入	录入平时成绩和考试成绩，暂存或提交	数据库触发器计算总评和绩点，提交后成绩只读
成绩修改申请	对已提交成绩填写修改原因	申请表保存原成绩快照，等待管理员审核
调课申请	选择目标星期、节次和教室	系统检查教室占用和教师时间冲突，待审核申请可撤销
考试修改申请	申请调整考试时间或教室	系统动态筛选可用教室，并记录原考试安排和申请后安排
退课审核与补考确认	审核学生退课，确认补考信息	教师意见写入申请表，后续由管理员继续处理
通知中心	查看公告和审批结果	通知按用户投递，支持已读状态维护

首先是教师端首页的展示，首页会如期显示基本信息，如果账户密码仍是默认密码，则会提示用户进行手动修改。同时，所有可以处理/查看的事项既可以通过卡片点击也可以通过顶侧菜单栏点击进行跳转。右上角可以快捷查看当前学期，同时可以点击个人名称下拉菜单选择修改密码或退出操作。如图 5 所示。

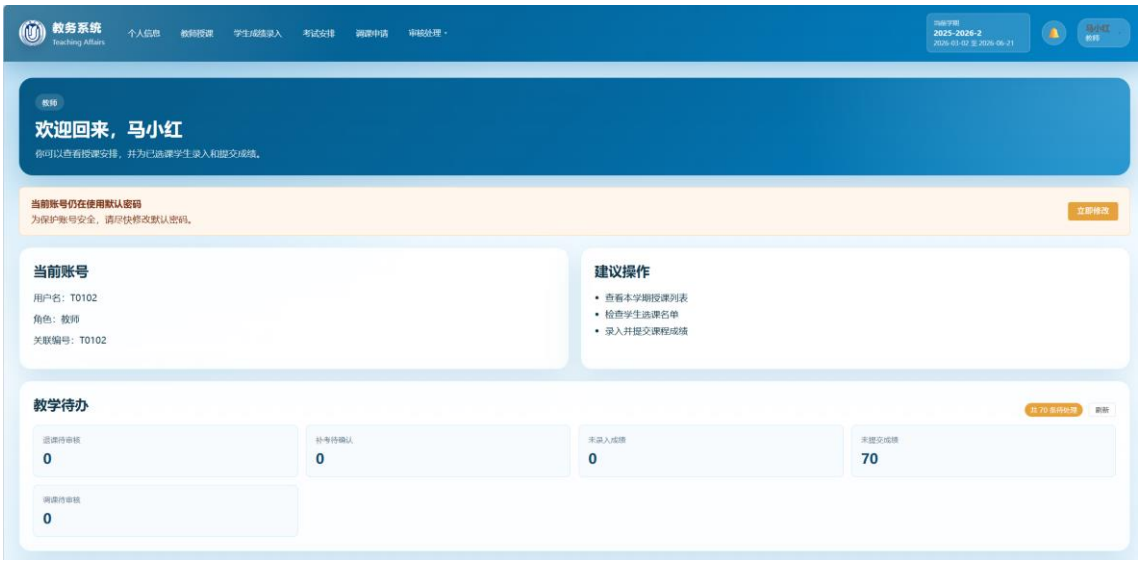


图 5 教师端首页展示

如图 6 所示，菜单栏点击“个人信息”，可以正常查看个人信息，其中，只有手机号和邮箱是可以自己更改的。



图 6 教师端个人信息修改页面

如图 7 所示，点击“教师授课”则可查看任意学期的授课课表，课表的右上角还会显示当前学期的状态，是否已经结束，是否是当前学期，是否是未开始的学期。注意到，每一门课下方都有一个查看详情，点击即可查看如图 8 所示的教学班详情，不但能够显示课程信息，还能显示班级成员信息，还可以方便的导出课表。

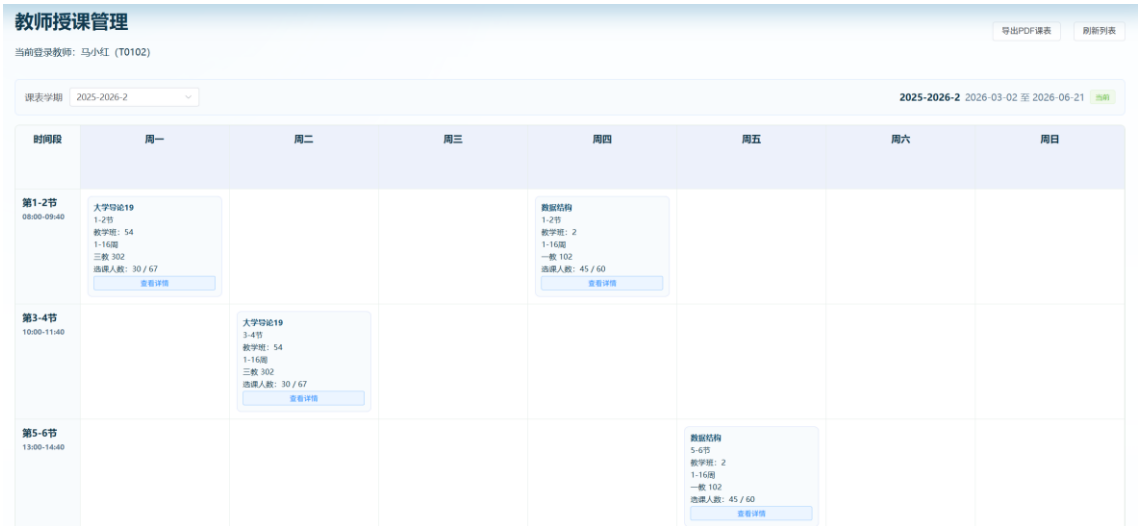


图 7 教师端课表查看



图 8 教师端课程详情查看

如图 9 所示，教师端可以完成成绩录入事务，可以选择对应的教学班进行筛选，查看班级目前的成绩分布，并且可以手动输入每个学生的平时/考试成绩，系统会根据该课程的二者权值计算出总评成绩和对应的绩点。其状态分为未提交和已提交，暂存并不会执行提交，但是能够自动计算对应的成绩和绩点。如果因为种种原因提交了错误的成绩，则可以向管理员申请修改，只需在对应的成绩后点击“申请修改”，管理员端即可收到请求，判断理由是否合理以决定是否修改，如图 11 所示。

为了方便成绩的离线统计和查看，该页面还提供了成绩导入和导出的功能，既支持程序易读的 csv 表格，也支持人类易操作的 xlsx 表格，如图 10 所示。

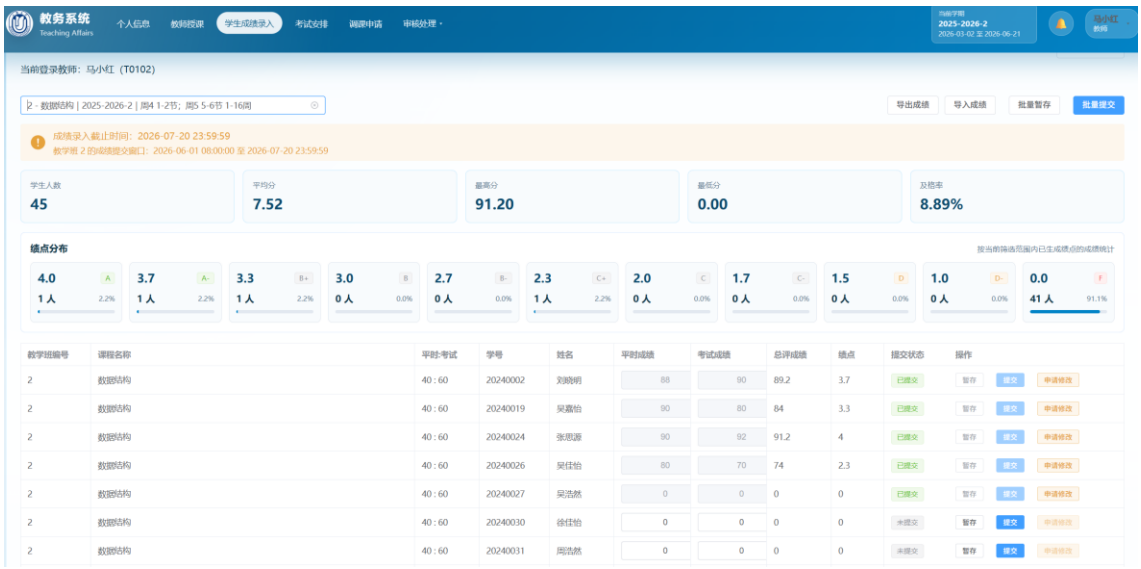


图 9 教师端学生成绩查看与录入

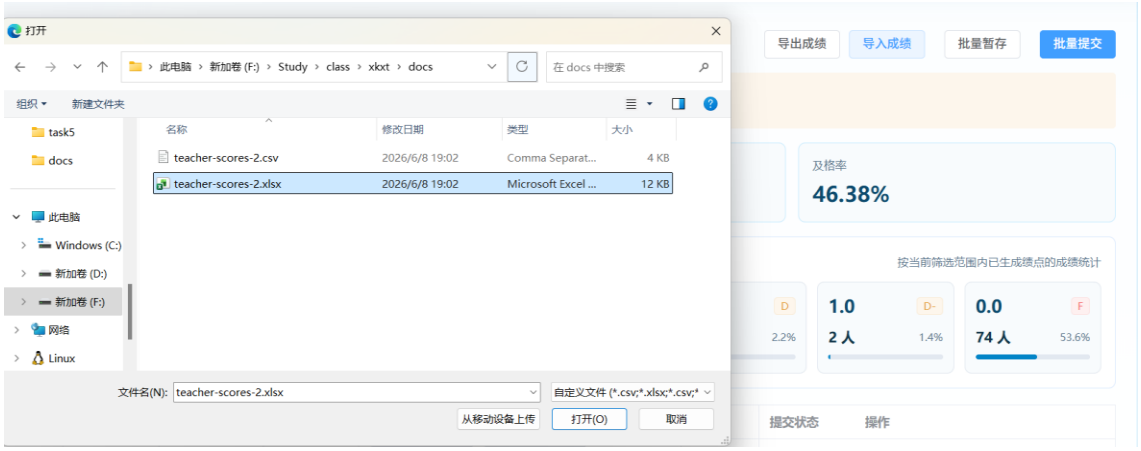


图 10 教师端成绩导入演示



图 11 已录入成绩的申请修改

当临近考试周时，管理员端可以排出考试安排表，此时教师端可以查看各教学班的考试时间和地点，同时可以自行根据具体情况，考虑“申请修改”，如图 12 所示。申请修改设置了冲突检测，当申请的时间与自己的其它教学班考试冲突，则不能申请，且申请的教室一定是当前没有被占用的，如图 13 所示。



图 12 考试安排查询

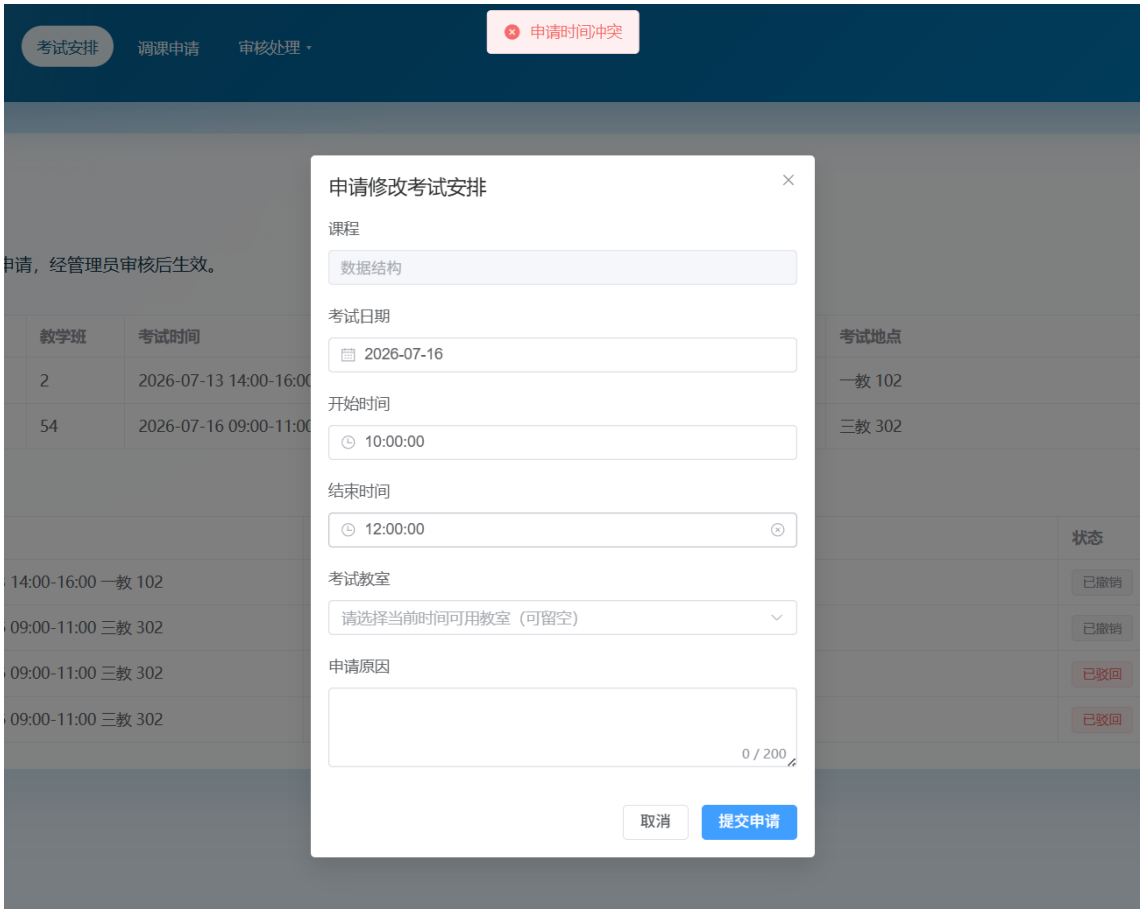


图 13 考试安排修改申请示例

如图 14 所示，对于教学班的教室/时间不满意的，教师也可以申请提出更改，考虑到默认情况下课程的时间是不允许修改的，因此当选择教学班之后默认的时间将和原来的时间一致，可以只选择更改教室。当然如果申请了冲突的时间或者教室，也有冲突检测来防止错误的申请，减轻了管理员的工作量。

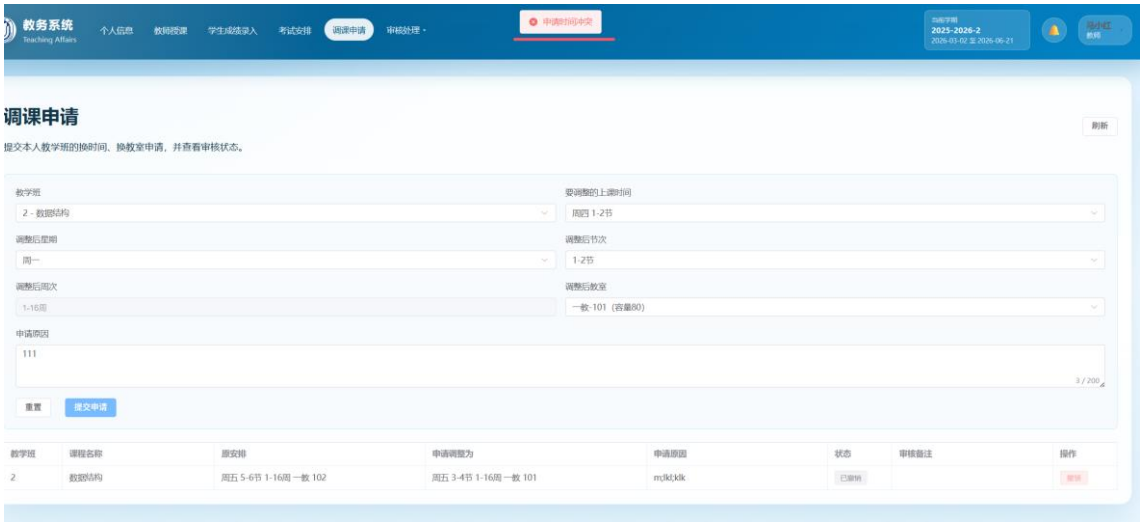


图 14 调课申请示例

如图 15、16 所示，教师端共有两个需要审核的申请，一个是学生的退课申请，一个是学生的补考申请，这二者的成功申请均需要经过教师和管理员的双端通过。如有申请，教师端会在首页待办显示，以提醒教师的审批处理。

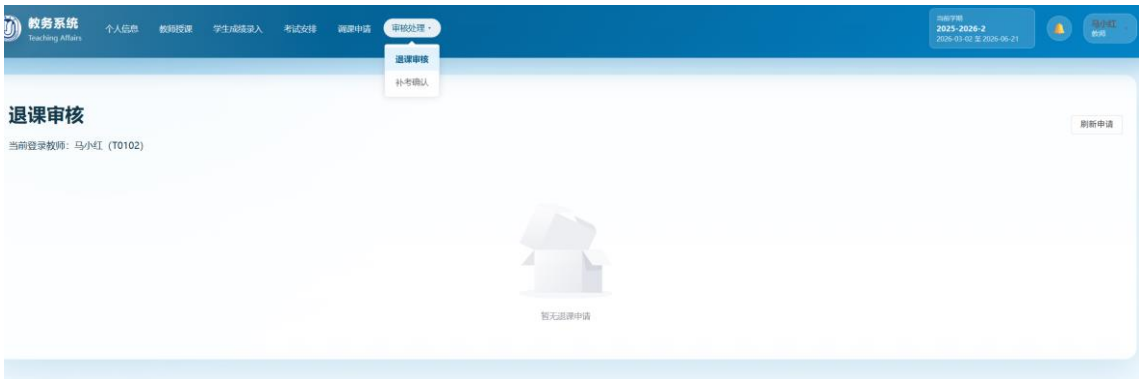


图 15 退课申请审核页面



图 16 补考申请审核页面

六、总结和评价

教师端的设计围绕教师每天真正会遇到的工作展开，包括看课表、看学生名单、录入成绩、提交成绩、申请调课、查看考试安排、处理退课和接收通知。相比单纯把这些功能做成几个页面，更重要的是把它们背后的数据关系理清楚。教师看到的不是孤立的课程，而是一个个教学班；教学班再连接课程、学期、教室、学生、成绩和考试安排。因此，教学班是教师端最核心的数据入口。

这次设计让我感受到，教师端看起来只是“教师使用的页面”，但实际上牵涉到很多跨角色的数据协作。学生选课之后，教师才能看到学生名单；教师提交成绩之后，学生才能查询成绩；教师申请调课或考试修改之后，还需要管理员审核；退课和补考也不是某一个角色单独完成的事情。也就是说，教师端并不是独立封闭的模块，而是连接学生端和管理员端的重要中间环节。

在实现上，我觉得最有价值的是把高风险操作放进流程里，而不是让教师直接修改最终数据。比如成绩提交后不能随便改，需要走成绩修改申请；调课不能直接改课表，需要先提交调课申请；考试安排也不能直接改，需要提交考试修改申请。这样虽然流程变长了一些，但能保留原因、原始数据、申请后的数据和审核意见，后面出现问题时也能查清楚。

成绩计算放在数据库触发器中也是比较合理的设计。教师只需要录入平时成绩和考试成绩，总评成绩和绩点由数据库统一计算。这样做可以减少前端、后端各算一遍带来的不一致，也能避免导入成绩时误填总评或绩点。对于成绩这种比较敏感的数据，规则统一比操作方便更重要。

这次设计也让我意识到，数据库设计不是简单地把字段列出来。比如教师端课表如果只在教学班表里放一个星期和节次，就很难支持一门课一周多次上课，也不好处理单双周安排。因此把

上课时间单独拆成教学班上课时间表，会让结构更清晰，也更方便后续扩展。再比如申请表看起来很多，但它们解决的是“过程可追踪”的问题，不是多余设计。

当然，教师端还有继续完善的空间。考试安排功能可以进一步增强，比如增加一键生成座位表，根据学生名单、考场容量和考试安排自动分配座位；也可以支持一个教学班对应多个考场，解决大班考试时单个教室容量不足的问题。成绩模块后续可以增加更细的成绩分析，例如按教学班查看分数段、异常成绩提醒、未提交成绩提醒等。通知模块也可以做得更主动，例如在成绩提交截止前提醒教师处理未提交成绩，在考试修改审核完成后及时提醒教师查看结果。

总体来说，教师端设计的重点是把教师工作流和数据库结构结合起来。页面负责让教师好用，后端负责把规则校验清楚，数据库负责把数据关系和关键约束保存下来。只有这三部分配合好，教师端才能不只是“能操作”，而是真正支撑教学过程中的数据准确、流程清晰和结果可追踪。